

Geração de mapas para jogos em 2D utilizando Redes Generativas Adversariais

Victor Le Roy Matos
Departamento de Computação
CEFET-MG
Belo Horizonte, Minas Gerais, Brasil
victorlrmatos@gmail.com

Rogério Martins Gomes
Departamento de Computação
CEFET-MG
Belo Horizonte, Minas Gerais, Brasil
rogerio@cefetmg.br

João Gabriel Gama Vila Nova
Departamento de Computação
UNIMA Afya
Maceió, Alagoas
jggvilanova@gmail.com

Resumo—Os jogos eletrônicos evoluíram de simples passatempos para uma forma de arte culturalmente relevante, impulsionados por avanços tecnológicos que permitiram gráficos mais realistas e uma jogabilidade mais imersiva. O presente trabalho explora o uso de Redes Gerativas Adversárias (GANs) na geração de mapas para jogos em 2D, com foco no jogo Super Mario Bros. A metodologia inclui a utilização de uma Wasserstein GAN (WGAN), combinada com um algoritmo de seleção de fragmentos e avaliação de jogabilidade realizada por um agente A*. Os mapas são gerados a partir de fragmentos obtidos de uma base de dados, refinados por critérios de qualidade que asseguram sua coesão estrutural e jogabilidade. Os resultados indicam uma melhoria significativa na qualidade dos mapas, com um aumento de 25% na taxa de fragmentos jogáveis e uma redução de 15% no tempo médio de avaliação pelo agente, garantindo maior aderência às características do jogo original. Este estudo contribui para o avanço da geração procedural de conteúdo em jogos, destacando a aplicabilidade das GANs em cenários de alta complexidade.

Palavras-chave—Game Map Generation, Generative Adversarial Networks, Super Mario Bros., Procedural Content Generation, Gameplay Evaluation

I. INTRODUÇÃO

Desde sua origem, os jogos eletrônicos evoluíram significativamente, passando de simples formas de entretenimento para uma expressão cultural e artística amplamente reconhecida [1]. Esse avanço é impulsionado por melhorias tecnológicas, como gráficos mais realistas, jogabilidade imersiva e inovações em inteligência artificial (IA), que permitem experiências interativas mais sofisticadas [2]. Além disso, a crescente popularização de comunidades *online* e plataformas de *streaming* ampliou o impacto cultural dos jogos [3] [4] [5].

Entre as várias áreas de inovação, a geração procedural de conteúdo (Procedural Content Generation - PCG) destaca-se por seu potencial de criar experiências únicas e personalizadas para os jogadores. Neste contexto, as Redes Geradoras Adversárias (GANs - Generative Adversarial Networks) emergiram como uma solução promissora, permitindo o desenvolvimento de mapas e níveis de jogos que respeitam aspectos críticos, como jogabilidade e *design* criativo [6] [7].

No entanto, a aplicação de GANs para jogos em 2D apresenta desafios específicos. Um deles é garantir que os mapas gerados sejam coerentes em termos de estrutura e jogabilidade. Outro desafio é a necessidade de respeitar as regras específicas de cada jogo, como *layout* de blocos e

distribuição de elementos interativos [8] [9]. Esses problemas tornam necessária a exploração de novas arquiteturas e de algoritmos que possam otimizar os resultados obtidos pelas GANs.

Dentre os jogos em 2D, Super Mario Bros. é frequentemente utilizado como objeto de estudo devido à sua combinação de simplicidade gráfica e complexidade estrutural [10] [11], que oferece um terreno ideal para experimentações com algoritmos de geração de níveis [12].

Este trabalho tem como objetivo principal investigar a viabilidade do uso de GANs para o desenvolvimento de mapas em jogos 2D. Além disso, propõe a incorporação de um algoritmo de seleção de *features* e um agente A* para avaliar os mapas gerados, com base em métricas de quantidade, jogabilidade e dificuldade. Espera-se que as soluções propostas possam aprimorar a qualidade e a validade dos níveis criados.

II. TRABALHOS RELACIONADOS

A geração procedural de conteúdo em jogos ganhou destaque a partir da competição de 2010 sobre Geração de Conteúdo Procedural (PCG), que utilizou Super Mario Bros. como foco de estudo [13] [14]. Essa iniciativa impulsionou diversos trabalhos acadêmicos, com foco em abordagens baseadas em Inteligência Artificial (IA), especialmente nas GANs, para a criação de níveis com atributos específicos.

Entre os estudos relevantes, destaca-se o TOAD-GAN [9], que utilizou escalas múltiplas para preservar características essenciais dos mapas, enquanto trabalhos baseados em Autoencoders Variacionais (VAEs) [15] demonstraram a possibilidade de combinar elementos de diferentes jogos. Comparativamente, as GANs mostram maior flexibilidade na criação de mapas que combinam diversidade com coesão estrutural.

No contexto das GANs, Silva, Torchelsen e Aguiar [16] compararam diferentes arquiteturas aplicadas à geração de mapas de masmorras, como cavernas, que respeitem as regras do jogo proposto pelos autores. Para garantir uma avaliação precisa, os autores padronizaram as condições experimentais, utilizando o mesmo *hardware*, métricas de avaliação e critérios para determinar a jogabilidade dos mapas. As arquiteturas analisadas incluíram VanillaGAN, Deep Convolutional GAN (DCGAN) e Wasserstein GAN (WGAN), com e sem penalidade de gradiente.

Os resultados demonstraram que GANs baseadas em camadas totalmente conectadas apresentaram resultados inferiores devido à maior suscetibilidade ao *overfitting*, comparado com arquiteturas baseadas em redes. Uma vantagem das convolucionais sobre as conectadas é que elas possuem flexibilidade no uso do gerador, que permite a criação de mapas de tamanhos diferentes ao utilizarem os vetores do espaço latente com diferentes dimensões. Além disso, os autores demonstraram que o treino de diferentes arquiteturas convolucionais e *loss functions* teve pouco impacto na melhora na geração dos níveis. No entanto, foi constatado que o uso de Spectral Normalization teve um efeito significativo na jogabilidade dos níveis gerados, contribuindo para uma maior coesão e realismo nos mapas produzidos, como visto na Tabela I.

Tabela I: Comparação entre o BatchNormalization e o SpectralNormalization aplicados em diversas arquiteturas. O resultado demonstra que o método SpectralNormalization gerou um aumento considerável na quantidade de níveis válidos [16].

Arquitetura	Regularização	Níveis válidos (%)
DCGAN	BatchNormalization	(mode collapse)
	SpectralNormalization	13,9
WGAN	BatchNormalization	7,4
	SpectralNormalization	11,4
WGANP	BatchNormalization	(mode collapse)
	SpectralNormalization	2,9

Ainda entre soluções baseadas em GANs, o modelo TOAD-GAN, proposto por Awiszus, Schubert e Rosenhahn [9] e fundamentado no trabalho de Shaham, Dekel e Michaeli [17], destacou-se ao introduzir uma abordagem que utiliza dados de treinamento em múltiplas escalas para preservar características essenciais dos mapas originais. Além disso, a TOAD-GAN propõe estratégias de *downsampling* para reduzir a perda de informação durante o treinamento, gerando mapas que combinam diversidade e coerência.

Durante o treinamento da TOAD-GAN, cada imagem fornecida foi subdividida em amostras de X escalas distintas. O valor de X foi estabelecido para permitir que os filtros convolucionais dos geradores e discriminadores abrangessem, na menor escala, metade da altura dos níveis, garantindo assim a modelagem adequada das estruturas mais altas.

Os autores também abordaram a questão da perda de informação durante o processo de *downsampling*. Eles observaram que a interpretação de cada *token* como um pixel de uma imagem e o subsequente *downsampling* ingênuo resulta em perda de informação, pois o *aliasing* poderia fazer com que *tokens* importantes desaparecessem em escalas menores. Para preservar a maior parte das informações do nível original, eles propuseram um método de *downsampling* que preserva a diversidade e a singularidade dos elementos gerados pelo modelo. Por exemplo, *tokens* que ocorrem com frequência e em múltiplos níveis, como os blocos de céu e terra, são considerados de menor importância do que *tokens* raros, como os blocos ocultos e especiais.

A TOAD-GAN foi avaliada utilizando quatro métricas

principais: avaliação qualitativa, Tile Pattern KL-Divergence (TPKL-Div), Level Embeddings e Level Authoring. A análise qualitativa revelou que 65% dos níveis gerados foram vencidos em testes realizados por um agente A* implementado conforme Baumgarten [18]. O TPKL-Div foi utilizado para avaliar a similaridade entre os mapas originais e os gerados, demonstrando que a TOAD-GAN produz valores mais baixos, o que reflete maior proximidade estrutural. A terceira métrica, Level Embeddings, evidenciou a capacidade do modelo de criar partes de mapas que não estão presentes no mapa original. Por fim, a métrica de Level Authoring mostrou que a TOAD-GAN consegue gerar mapas autorais, pois consegue editar os *tokens* em qualquer tipo de mapa e em qualquer escala.

Em estudos subsequentes, Schrum et al. [8] [19] demonstraram a eficácia das GANs na geração de mapas para jogos. No primeiro estudo [8], os autores propuseram uma metodologia para criar mapas com atributos específicos, como níveis jogáveis no jogo Super Mario Bros., que otimizam o número de saltos necessários para a conclusão do nível.

A metodologia combinou características de uma DCGAN com representações de um espaço latente, utilizando estratégias de treinamento inspiradas no trabalho de Awiszus, Schubert e Rosenhahn [9].

A avaliação incluiu duas métricas principais: Teste Baseado em Representação, que analisou a distribuição dos blocos no mapa, e Teste Baseado em Agente, que verificou a jogabilidade utilizando um agente A* implementado por Shaker [13]. Apesar dos desafios encontrados na definição de uma função de adaptabilidade (*fitness function*) eficiente para capturar as características desejadas, os resultados demonstraram que a combinação de GANs com espaços latentes é capaz de gerar uma ampla variedade de níveis com múltiplas soluções.

No segundo estudo, os autores [19] concentraram-se na evolução interativa de mapas para os jogos Super Mario Bros. e The Legend of Zelda. Essa abordagem contou com a colaboração de usuários por meio de uma interface gráfica, utilizando Wasserstein GANs (WGANs) para gerar níveis. Os vetores de entrada foram ajustados para cada jogo: 5 entradas e 13 camadas de profundidade para Super Mario Bros., e 10 entradas e 3 camadas de profundidade para The Legend of Zelda. As amostras foram extraídas do Video Game Level Corpus (VGLC) e segmentadas em matrizes de dimensões fixas.

A metodologia de evolução interativa de mapas foi organizada em três estratégias distintas. A primeira estratégia, Seleção Seletiva, foi implementada utilizando um algoritmo elitista que permite aos usuários selecionar os resultados preferidos para a evolução subsequente. Apesar de eficaz, essa abordagem pode, ocasionalmente, induzir a convergência do algoritmo para uma região específica do espaço de busca, comprometendo a diversificação dos mapas evoluídos. A segunda estratégia, Exploração Manual do Espaço Latente, permite que os usuários manipulem diretamente os valores no espaço latente por meio de uma interface gráfica com controles deslizantes, resultando em uma visualização e evolução imediatas

do mapa. A terceira estratégia, Modificação do Conteúdo Gerado, permite que os usuários alterem características específicas dos mapas gerados, como o aumento do tamanho de uma fase em Super Mario Bros. ou a configuração das masmorras em The Legend of Zelda.

Assim como as GANs, os Autoencoders Variacionais (VAEs) também têm sido explorados na geração de mapas. Sarkar e Cooper [20] exploraram essa funcionalidade das VAEs, demonstrando que é possível gerar mapas que não apenas satisfazem propriedades desejadas, como também combinam elementos de diferentes jogos. Especificamente, uma VAE foi treinada com níveis de Super Mario Bros. e Kid Icarus, aprendendo uma representação latente que abrange ambos os jogos. Essa representação é utilizada para gerar segmentos de mapa que incorporam características misturadas dos dois jogos.

Os autores adotaram uma metodologia semelhante à dos trabalhos anteriores, com jogos de plataforma em orientações distintas (vertical e horizontal). Os mapas foram codificados usando One-Hot encoding e segmentados em quadrados de 16x16 pixels. A aprendizagem desse tipo de segmento pela VAE permite que as criações de mapas sejam divididas por segmentos, facilitando uma abordagem de *design* conduzida e co-criativa, na qual os *designers* podem selecionar e combinar segmentos conforme desejado. Além disso, a geração também é capaz de utilizar espaços latentes aleatórios ou matrizes de covariância para evoluir o mapa com base nas preferências do *designer*.

Para avaliar os dados, os autores definiram quatro métricas principais. A primeira, densidade, mede o número de blocos sólidos que colidem com o jogador em cada segmento. A segunda, dificuldade, avalia o total de inimigos presentes em um segmento, sendo 16 o valor máximo. A terceira, não-linearidade, analisa o quão próximo o segmento gerado está de uma linha reta, com base nos blocos do chão. A quarta métrica calcula a proporção de blocos de cada jogo (Mario e Kid Icarus) em segmentos combinados. Comparando VAE, GAN e VAE-GAN, os resultados indicaram que VAEs capturam melhor os espaços latentes, integrando características dos jogos com maior eficiência.

No segundo trabalho [21], Sarkar e Cooper ampliaram sua abordagem para gerar níveis completos e contínuos, com uma variedade de características nos mapas, como progressão em diferentes direções, enquanto lidavam com um tamanho fixo de espaço latente. A geração contínua foi alcançada ao decodificar as características latentes de segmentos subsequentes, garantindo a progressão lógica do nível.

A metodologia de geração foi semelhante à utilizada no trabalho anterior, porém com a adição de um gerador de segmentos sequenciais e um classificador para avaliar a melhor posição para adicionar o segmento gerado. As métricas para avaliação dos resultados foram mais bem definidas, incluindo a descontinuidade, que verifica se os segmentos foram bem conectados para os mapas obterem um maior senso de progressão e a mistura do nível com outros jogos. Outras submétricas foram definidas, como a densidade e a não-linearidade.

Apesar desses avanços, desafios permanecem, como a criação de níveis que respeitem regras específicas de *design* e a avaliação automatizada da qualidade dos mapas gerados. Este estudo busca preencher lacunas identificadas, como a dificuldade de gerar mapas coerentes e jogáveis em jogos 2D. A proposta difere ao combinar GANs com algoritmos de seleção de features e avaliação baseada em agentes, abordando limitações de métodos anteriores e garantindo maior validade estrutural.

III. METODOLOGIA

A metodologia deste estudo, apresentada na Figura 1, foi desenvolvida para garantir a eficiência e a precisão na geração de mapas 2D para jogos, utilizando GANs e algoritmos de seleção. Esta seção detalha cada etapa do processo, desde a obtenção dos dados até a avaliação dos mapas gerados.

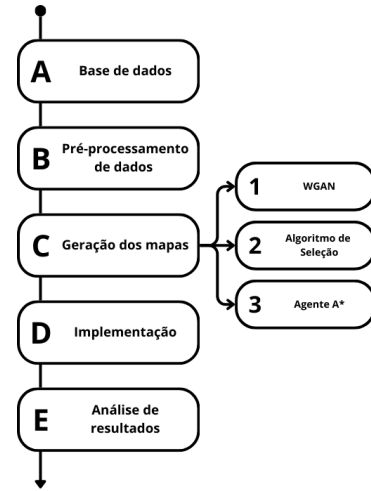


Figura 1: O fluxograma descreve a metodologia para geração e avaliação de mapas em jogos 2D, envolvendo coleta e pré-processamento de dados, uso de WGANs, seleção e avaliação com o algoritmo A*, seguida de implementação e análise de resultados.

A. Base de Dados

Para este estudo, utilizou-se a base de dados Video Game Level Corpus (VGLC) [22], que fornece mapas de diferentes jogos traduzidos em representações textuais por meio de abordagens distintas: blocos, grafos e vetores. Optou-se pelas representações baseadas em blocos devido à facilidade de conversão para imagens e compatibilidade com redes neurais convolucionais. Os mapas baseados em blocos são bidimensionais, em que cada caractere nos arquivos de texto representa um bloco do jogo, como ilustrado na Figura 2. Um arquivo JSON é fornecido para mapear cada caractere ao bloco correspondente na *spritesheet* (Figura 3). As dimensões mínimas, médias e máximas dos mapas analisados estão resumidas na Tabela II.

Tabela II: Dimensões dos mapas analisados.

Mapas	Tamanho		
	Mínimo	Médio	Máximo
150	150x14	188x14	374x14

B. Pré-processamento de dados

A fase de pré-processamento, fundamental para garantir dados limpos e estruturados, incluiu a remoção de inconsistências, normalização dos valores e transformação dos dados para formatos compatíveis com os modelos de aprendizado de máquina. São elas:

1) *Label Encoding*: Codificou-se cada bloco utilizando valores numéricos para padronizar a entrada da rede. Para o jogo Super Mario Bros., os blocos foram codificados conforme a Tabela III.

Tabela III: Codificação de blocos do jogo.

Bloco	Símbolo	Identidade	Visualização
Bloco sólido/Chão	X	0	
Bloco quebrável	S	1	
Espaço vazio (passável)	-	2	
Bloco de interrogação cheio	?	3	
Bloco de interrogação vazio	Q	4	
Inimigo	E	5	
Cano superior esquerdo	<	6	
Cano superior direito	>	7	
Cano esquerdo	[	8	
Cano direito	]	9	

2) *Divisão em Amostras*: Os mapas foram subdivididos em amostras menores para ampliar o conteúdo de treinamento. Esse processo foi realizado ao deslizar uma janela de 28x14 blocos sobre o mapa original do jogo. Nas redes, as amostras possuem tamanho ajustado de 32x32. Para atender a essa restrição, as amostras foram ampliadas com blocos vazios, garantindo a manutenção dos dados principais e sua adaptação ao formato de entrada exigido pelas redes.

O foco da geração são pequenas amostras, ou fragmentos, que representam partes do mapa. Essa abordagem facilita a união dessas partes ao final do processo de criação, permitindo a construção de mapas completos para o jogo Super Mario Bros.

C. Geração de mapas

A geração de mapas é uma etapa que integra aprendizado profundo e técnicas de otimização para produzir níveis coesos e jogáveis. O processo inicia-se com o uso das GANs para criação dos fragmentos de mapas. Após essa etapa, os fragmentos passam por um refinamento conduzido por algoritmos de seleção, que asseguram a qualidade e a jogabilidade das amostras. Por fim, os mapas completos são submetidos a avaliações detalhadas, verificando funcionalidade e dificuldade, garantindo sua adequação aos requisitos específicos do jogo-alvo.

1) *Redes Geradoras Adversárias (GANs)*: Optou-se pela Wasserstein GAN (WGAN) [23], exibida na Figura 4, devido à sua estabilidade em treinamento e capacidade de minimizar problemas como colapso de modos. A arquitetura inclui:

- Gerador: Camadas com ReLU e batch normalization para estabilizar o treinamento.

- Critic (Discriminador): Utiliza Leaky ReLU e batch normalization para avaliar as amostras geradas.

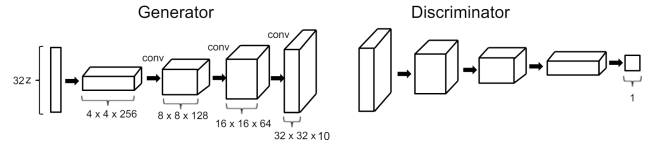


Figura 4: Arquitetura utilizada na WGAN.

Os autores usam o termo *critic* para o discriminador, uma vez que ele não é treinado para classificar, mas para favorecer os ajustes do gerador e tentar minimizar a distância entre a amostra real e o dado gerado. O gerador, por sua vez, tenta aproximar as distâncias entre as amostras reais com as geradas [16].

O vetor de entrada do gerador possui dimensão 32, ou vetor de espaço latente, e o *output* final é uma matriz 32x32x10, representando os blocos do jogo. As camadas intermediárias representam uma série de camadas convolucionais para produzir a saída com a dimensão correta [24] [25].

O discriminador recebe como entrada um mapa gerado pelo gerador (ou um real) de dimensão 32 x 32 x 10 e o processa para classificar se a entrada é real ou falsa. As camadas intermediárias representam uma série de camadas convolucionais que reduzem progressivamente a dimensão da entrada. A saída final é um único valor escalar (1), que indica a probabilidade de a entrada ser real ou falsa [24] [25].

Neste estudo, as amostras geradas correspondem a fragmentos de mapas com dimensões de 28x14 blocos, que serão posteriormente combinados para formar mapas completos. A escolha por gerar fragmentos, em vez de mapas completos, foi fundamentada nos seguintes fatores:

- Aumento da Diversidade de Dados**: Durante a etapa de pré-processamento, o mapa original foi dividido em amostras de 28x14 blocos, com o objetivo de ampliar o conjunto de dados disponíveis para o treinamento. Essa abordagem também influenciou o design da arquitetura da rede, que foi projetada para receber uma amostra como entrada e reproduzi-la como saída.
- Simplicidade e Estruturação**: A geração de fragmentos reduz a complexidade do algoritmo, uma vez que o tamanho fixo facilita a construção do modelo e melhora a consistência das partes geradas.
- Flexibilidade e Reutilização**: Fragmentos gerados podem ser reutilizados em diferentes posições e combinações, o que permite a criação de mapas variados a partir de um mesmo conjunto de amostras.

No total, foram gerados 10.000 fragmentos, que serviram como base para os processos subsequentes de montagem e avaliação de mapas completos.

2) *Algoritmo de seleção*: Após a geração dos fragmentos, o algoritmo de seleção é responsável por identificar aqueles que atendem a critérios específicos de qualidade, com base na coerência estrutural. Esse processo garante que os blocos

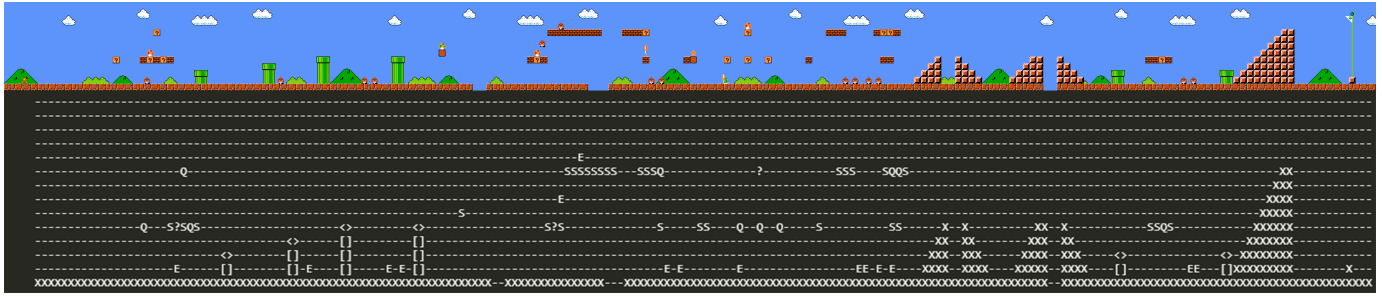


Figura 2: Representação do mapa do jogo Super Mario Bros. em formato de texto.

```
{
  "tiles": {
    "x": ["solid", "ground"],
    "s": ["solid", "breakable"],
    ".": ["passable", "empty"],
    "?": ["solid", "question block", "full question block"],
    "Q": ["solid", "question block", "empty question block"],
    "e": ["enemy", "damaging", "hazard", "moving"],
    "<": ["solid", "top-left pipe", "pipe"],
    ">": ["solid", "top-right pipe", "pipe"],
    "[": ["solid", "left pipe", "pipe"],
    "]": ["solid", "right pipe", "pipe"],
    "o": ["coin", "collectable", "passable"],
    "B": ["solid", "bullet bill", "hazard", "enemy"],
    "b": ["solid", "bullet bill"]
  }
}
```

Figura 3: JSON de tradução dos *tokens* para os elementos da *spritesheet* com base em tags.

gerados respeitem as regras do jogo, como o posicionamento e a construção adequada de canos, assegurando a jogabilidade. Para isso, o algoritmo seleciona os 1.000 melhores fragmentos, correspondentes a 10% do total gerado, que serão utilizados na criação dos mapas completos.

A seleção é guiada por uma função de adaptabilidade de minimização, definida por três critérios que foram agregados a uma única função *fitness* que será apresentada posteriormente. Os principais critérios considerados são:

a) Avaliação de Canos: A qualidade dos canos é avaliada considerando:

- O número de canos dentro de um fragmento, que deve estar dentro de um intervalo predefinido;
- Penalizações aplicadas para blocos de canos inseridos em locais inadequados.

A métrica de *fitness* para canos é calculada pela (Equação 1):

$$C_{fit} = |Cnum - Cmax| \times \alpha + |Cnum - Cmin| \times \beta + \sum_0^N \gamma, \quad (1)$$

em que:

- *Cnum*: número de canos bem construídos no fragmento;
- *Cmin*, *Cmax*: número mínimo e máximo de canos permitidos;
- *N*: número de canos mal construídos ou bloco de canos mal posicionados no fragmento;
- α, β, γ : coeficientes de penalização.

b) Avaliação de Inimigos: A distribuição e o posicionamento dos inimigos são verificados considerando:

- O número de inimigos em um fragmento, que deve estar dentro de um intervalo especificado;
- Penalizações para inimigos posicionados incorretamente.

A métrica de *fitness* para inimigos é calculada pela (Equação 2):

$$I_{fit} = |Inum - Imax| \times \delta + |Inum - Imin| \times \epsilon + \sum_0^N \zeta, \quad (2)$$

em que:

- *Inum*: número de inimigos encontrados no fragmento;
- *Imin*, *Imax*: número mínimo e máximo de inimigos permitidos;
- *N*: número de inimigos mal posicionados;
- δ, ϵ, ζ coeficientes de penalização.

c) Avaliação de Buracos: A quantidade e o posicionamento dos buracos são avaliados considerando:

- O número de buracos dentro de um fragmento, que deve estar dentro de um intervalo definido.

A métrica de *fitness* para buracos é calculada pela (Equação 3):

$$B_{fit} = |Bnum - Bmax| \times \theta + |Bnum - Bmin| \times \lambda, \quad (3)$$

em que:

- *Bnum*: número de buracos bem construídos no fragmento;
- *Bmin*, *Bmax*: número mínimo e máximo de buracos permitidos;
- θ, λ : coeficientes de penalização.

Por padrão, todos os coeficientes de penalização foram definidos empiricamente com o valor de 0,2. Esse valor foi determinado por meio de execuções iterativas do processo de seleção, nas quais os hiperparâmetros foram ajustados e os resultados avaliados até que os fragmentos selecionados pelo algoritmo estivessem em um padrão considerado adequado para as regras do jogo. A função de adaptabilidade final é calculada como a soma das métricas individuais (Equação 4):

$$fitness = C_{fit} + I_{fit} + B_{fit} \quad (4)$$

Esse processo de seleção garante que apenas os fragmentos com maior qualidade estrutural e aderência às regras do jogo sejam considerados para as etapas subsequentes.

3) *Avaliação por agente A**: Após a conclusão das etapas de geração e seleção, os fragmentos foram avaliados por um agente A*, projetado para completar mapas no menor tempo possível, conforme descrito por Khalifa et al. [14]. O objetivo dessa etapa foi garantir que os mapas gerados fossem funcionalmente jogáveis. O processo seguiu os seguintes passos:

a) *Validação de Fragmentos*: Os fragmentos gerados foram organizados e submetidos ao agente A* para testes individuais de jogabilidade. Cada teste foi limitado por um tempo de 60 segundos para cada fragmento e, caso o tempo seja ultrapassado, o teste falha. Em seguida, foram divididos em dois grupos:

- Grupo 1: Contém apenas os fragmentos vencidos pelo agente A* dentro do limite de tempo estipulado.
- Grupo 2: Inclui todos os fragmentos, independentemente de terem sido vencidos ou não.

Essa separação permitiu comparar a quantidade e a qualidade dos mapas gerados, avaliando se o uso exclusivo de fragmentos vencidos contribui para mapas mais consistentes e bem estruturados.

b) *Construção de Mapas Jogáveis*:

- *Teste Inicial de Jogabilidade*: Cada fragmento do grupo selecionado é adicionado, um a um, a um *array* que formou um mapa expandido.
- *Validação do Mapa Expandido*: A cada novo fragmento concatenado ao mapa, o agente avaliava, em um período máximo de 120 segundos, a jogabilidade do conjunto. Caso o mapa permanecesse funcional, o processo continuava. Caso contrário, o último fragmento adicionado era removido e outra combinação era testada.
- *Finalização*: O mapa era considerado completo ao atingir o limite de 10 fragmentos concatenados e permanecer jogável. O mapa gerado era salvo e o processo era reiniciado com novos fragmentos.

D. Implementação

A implementação foi conduzida em Python utilizando a biblioteca PyTorch, sendo estruturada para atender às especificidades do treinamento e avaliação da GAN. O pré-processamento incluiu a criação de amostras específicas para o treinamento, configuradas com base nos parâmetros descritos por Schrum et al. [8] [19]. A configuração da rede incluiu o otimizador RMSProp, uma taxa de aprendizado de 0,00005 e a função de erro Wasserstein Loss [26].

Durante o treinamento, foram gerados fragmentos de mapas de dimensão 32x32, os quais foram posteriormente traduzidos para os blocos correspondentes de cada jogo. Após essa etapa, os fragmentos passaram pelo algoritmo de seleção e, em seguida, foram avaliados pelo agente A* para validação e construção final dos mapas completos.

Após a construção dos mapas, foi realizada uma análise detalhada dividida em dois aspectos principais:

1) *Análise Estatística*: Foram calculadas a média e o desvio padrão de diversos fatores relacionados aos mapas, incluindo:

- Tempo gasto para finalizar o mapa;
- Número total de saltos;
- Número total de inimigos derrotados;
- Dificuldade geral do mapa.

A dificuldade foi calculada utilizando a Equação 5:

$$D = N_{tempoGasto} \times \omega_t + N_{totalSaltos} \times \omega_p + N_{inimigosEliminados} \times \omega_i, \quad (5)$$

em que:

- $N_{tempoGasto}$, $N_{totalSaltos}$, $N_{inimigosEliminados}$: representam a normalização dos valores tempo gastos, número total de saltos e inimigos eliminados, respectivamente;
- ω_t , ω_p , ω_i : pesos atribuídos aos fatores tempo, 0, 3, saltos, 0, 3, e inimigos derrotados, 0, 4, respectivamente. A soma dos pesos é sempre igual a 1

Os pesos atribuídos foram definidos empiricamente por meio de testes e análises dos resultados do algoritmo. Observou-se que o tempo e os saltos estão relacionados, devido à duração total de execução dos saltos. Por essa razão, esses fatores recebem pesos menores individualmente, mas, quando somados, correspondem a mais de 50% do peso total. Por outro lado, os inimigos, cujas influências dependem da quantidade presente no mapa, recebem o peso restante para que a soma totalize 1. Além disso, os mapas gerados foram classificados em faixas de dificuldade (fácil, médio e difícil), permitindo uma categorização mais precisa e a análise do impacto das escolhas metodológicas na criação dos níveis.

2) *Análise Empírica*: Comparação qualitativa entre os mapas gerados diretamente pela GAN e aqueles refinados pelo algoritmo de seleção, com o objetivo de identificar ganhos em coerência estrutural e qualidade de construção dos mapas.

IV. ANÁLISE DOS RESULTADOS

Esta seção analisa os resultados do processo de geração de mapas, que são compostos por uma união de 10 fragmentos, para determinar se a GAN, assistida pelo agente A*, pode gerar mapas jogáveis do Super Mario Bros. Além disso, também é avaliado o impacto da incorporação de um algoritmo de seleção na melhoria da qualidade dos mapas gerados.

A. Análise Estatística

Dois experimentos foram conduzidos para avaliar o desempenho da WGAN com e sem um algoritmo de seleção.

1) *WGAN*: Neste experimento, os resultados foram obtidos por meio do processo de geração de 10.000 fragmentos utilizando a GAN, sem a aplicação de algoritmo de seleção, que foram montados e avaliados pelo método de agrupamento:

- Grupo 1: Mapas montados apenas com fragmentos jogáveis (vencidos pelo agente A* e dentro do limite de 60 segundos);
- Grupo 2: Mapas montados com todos os fragmentos gerados.

As Tabelas IV e V apresentam uma síntese estatística dos dados coletados.

Tabela IV: Quantidade de mapas gerados (vencidos pelo A* e dentro do limite de 120 segundos) durante o teste da WGAN, divididos por faixa de dificuldade.

	Fácil (0 - 0,3)	Médio (0,3 - 0,7)	Difícil (0,7 - 1)	Total
Grupo 1	150	25	1	176
Grupo 2	159	27	1	187

A Tabela IV classifica os mapas gerados em três faixas de dificuldade:

- Fácil: [0 a 0,3);
- Médio: [0,3 a 0,7);
- Difícil: [0,7 a 1].

a) Distribuição de Dificuldade: A maioria dos mapas em ambos os grupos foram classificados como Fácil (escores de dificuldade entre 0 e 0,3). Isso pode ser atribuído à eficácia do agente A* na conclusão dos mapas ou à conexão de fragmentos complexos ou mal estruturados com outros fragmentos já vencidos, reduzindo a dificuldade geral.

b) Impacto de Fragmentos Não Jogáveis: A inclusão de fragmentos não jogáveis no Grupo 2 resultou em uma pequena diferença no número total de mapas gerados e não alterou significativamente a proporção entre os níveis de dificuldade.

c) Taxa de Conclusão de Mapas: O Grupo 1 gerou mapas a partir de 17,6% dos fragmentos, enquanto o Grupo 2 utilizou 18,7% dos fragmentos. Mesmo com menos fragmentos, o Grupo 1 manteve um número comparável de mapas gerados, indicando que os fragmentos jogáveis eram mais bem estruturados.

Tabela V: Resultados estatísticos para a geração utilizando apenas a WGAN pura. A tabela apresenta os valores máximos, mínimos, médios e o desvio padrão de cada fator, ao avaliar todos os mapas gerados de um grupo.

	Tempo gasto (s)				Inimigos eliminados			
	Máximo	Mínimo	Média	Desvio Padrão	Máximo	Mínimo	Média	Desvio Padrão
Grupo 1	43	12	13,4	2,32	17	1	8,04	2,91
Grupo 2	117	12	14,35	9,06	15	1	7,92	2,84

	Saltos				Dificuldade (0 - 1)			
	Máximo	Mínimo	Média	Desvio Padrão	Máximo	Mínimo	Média	Desvio Padrão
Grupo 1	80	12	29,6	5,5	0,8	0,048	0,238	0,081
Grupo 2	221	12	22,45	15,64	0,714	0,014	0,22	0,09

A Tabela V organiza os dados em quatro métricas principais: tempo gasto; inimigos eliminados; saltos; e dificuldade. A análise dessas métricas levou às seguintes conclusões:

a) Tempo de Conclusão: O Grupo 1 apresentou um tempo médio de conclusão menor e menor variação em relação

ao Grupo 2, o que sugere uma experiência de jogo mais consistente devido ao menor desvio padrão;

b) Inimigos Eliminados: A média e o desvio padrão do número de inimigos eliminados foram ligeiramente maiores no Grupo 1. No entanto, essa diferença provavelmente não é significativa. Assim, não é possível determinar com certeza qual grupo apresentou melhor desempenho em relação à geração;

c) Saltos: O Grupo 1 exigiu um número médio maior de saltos, enquanto o Grupo 2 apresentou maior variação e valores máximos mais elevados. Os resultados encontrados no Grupo 2 podem indicar inconsistências na inclusão de mapas não jogáveis, como conexões inadequadas ou fragmentos mal estruturados, uma vez que o desvio padrão foi consideravelmente superior comparado ao Grupo 1;

d) Dificuldade Geral: Os escores médios de dificuldade e suas variações foram comparáveis entre os grupos, indicando que o agente A* manteve níveis de dificuldade semelhantes mesmo com a inclusão de fragmentos não jogáveis.

2) *Algoritmo de Seleção*: No primeiro experimento, foram incluídos exemplos de mapas que possuem erros estruturais, como canos sobrepostos ou buracos inalcançáveis, destacando limitações do modelo. A aplicação do algoritmo de seleção eliminou a maioria desses problemas. Dessa forma, o segundo experimento utilizou o algoritmo de seleção para selecionar, dentre os 10.000 mapas utilizados no experimento anterior, 1.000 fragmentos de mapas, que foram montados e avaliados pelo mesmo método de agrupamento do experimento anterior.

Tabela VI: Quantidade de mapas gerados (vencidos pelo A* e dentro do limite de 120 segundos) durante o teste do algoritmo de seleção, divididos por faixa de dificuldade.

	Fácil (0 - 0,3)	Médio (0,3 - 0,7)	Difícil (0,7 - 1)	Total
Grupo 1	36	27	1	64
Grupo 2	11	9	1	21

A Tabela VI classifica os mapas gerados nas mesmas faixas de dificuldade da Tabela IV, enquanto a Tabela VII apresenta uma análise estatística das características dos quadros gerados, assim como a Tabela V. Os principais resultados obtidos foram:

a) Distribuição de Dificuldade: O Grupo 1 apresentou a maioria dos mapas como “fáceis” (56% do total), enquanto o Grupo 2 exibiu uma distribuição mais equilibrada;

b) Impacto do Algoritmo de Seleção: No Grupo 2, a inclusão do algoritmo resultou em uma menor quantidade de mapas jogáveis, mas manteve as proporções entre os níveis de dificuldade. Essa quantidade inferior de mapas gerados sugere que os fragmentos não-jogáveis dificultaram a geração de mapas, uma vez que há a chance de uma parte não-jogável gerar um *deadlock* no processo. Um exemplo é quando um fragmento não jogável que ultrapassou o limite de tempo é inserido no mapa e é aprovado. Isso

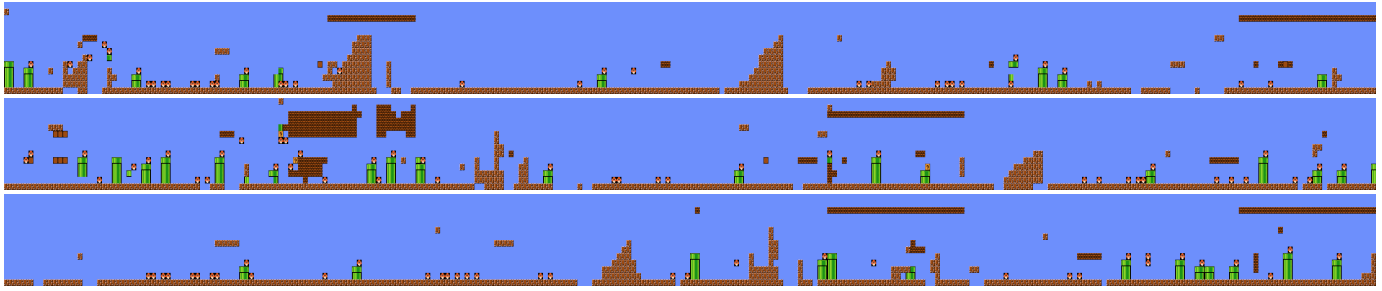


Figura 5: Mapas do jogo Super Mario Bros. gerados através da WGAN pura. Os mapas representam, de cima para baixo, as faixas fácil, médio e difícil, respectivamente.

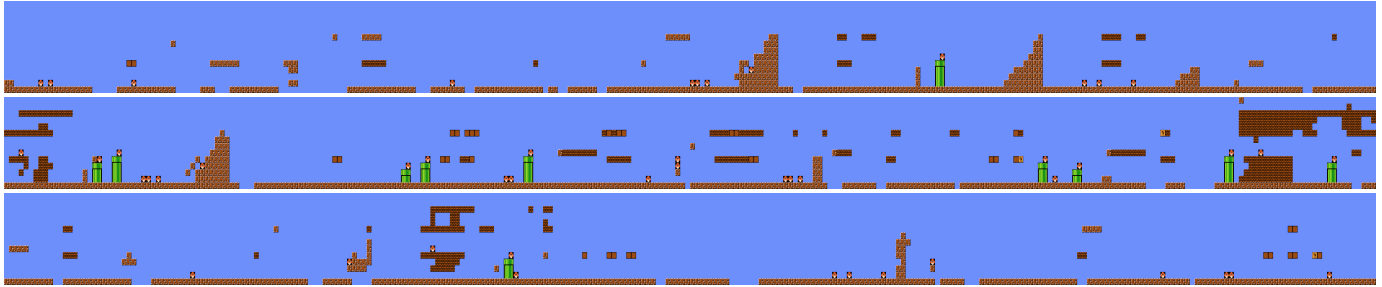


Figura 6: Mapas do jogo Super Mario Bros. gerados através da WGAN combinada ao algoritmo de seleção. Os mapas representam, de cima para baixo, as faixas fácil, médio e difícil, respectivamente.

pode fazer com que, nos próximos trechos inseridos, o tempo de 120 segundos seja ultrapassado.

- c) Taxa de Conclusão de Mapas: O Grupo 1 alcançou uma taxa de sucesso de 64% na geração de mapas jogáveis, enquanto o Grupo 2 obteve 21%.

Tabela VII: Resultados estatísticos para a geração utilizando a WGAN combinada ao algoritmo de seleção. A tabela apresenta os valores máximos, mínimos, médios e o desvio padrão de cada fator, ao avaliar todos os mapas de um grupo.

	Tempo gasto (s)				Inimigos eliminados			
	Máximo	Mínimo	Média	Desvio Padrão	Máximo	Mínimo	Média	Desvio Padrão
Grupo 1	26	12	13,2	1,8	10	1	4,5	2,5
Grupo 2	28	12	13,57	3,26	8	1	4,28	1,95
	Saltos				Dificuldade (0 - 1)			
	Máximo	Mínimo	Média	Desvio Padrão	Máximo	Mínimo	Média	Desvio Padrão
Grupo 1	36	10	19,5	0,14	0,91	0,069	0,29	0,142
Grupo 2	47	10	20,24	6,67	0,77	0,08	0,3	0,153

- d) Tempo de Conclusão: O resultado obtido pelos Grupos foram similares. Apesar disso, os resultados apontam que o algoritmo ajudou a selecionar fragmentos mais bem estruturados, pois manteve a consistência dos dados;
- e) Inimigos Eliminados: Os resultados foram similares em ambos os grupos, porém o Grupo 1 obteve um desvio padrão ligeiramente superior ao Grupo 2, o que sugere que, ao utilizar os fragmentos jogáveis, há uma maior variabilidade nos desafios de jogo dentro dos mapas;
- f) Saltos: O Grupo 1 apresentou um número médio menor de saltos, enquanto o Grupo 2 teve o desvio padrão consideravelmente maior e valores máximos mais altos. Ao analisar os dados destes 21 mapas gerados, é constatado

que os *outliers* tiveram grande impacto no valor do desvio padrão, pois, ao remover o maior e o menor valor, o resultado caiu para 2,30. Dessa forma, essa variação está relacionada a menor quantidade de mapas gerados;

- g) Dificuldade Geral: Os níveis médios de dificuldade foram semelhantes, mas o Grupo 2 apresentou maior variação. Esses resultados sugerem que, mesmo com a inclusão de fragmentos não jogáveis, o agente A* concluiu os mapas mantendo níveis comparáveis de dificuldade.

B. Análise Empírica

Com base nas análises anteriores, constatou-se que os fragmentos jogáveis são mais adequados para a geração de mapas de maior qualidade. Por essa razão, apenas os mapas compostos por esses fragmentos foram considerados nesta análise. Para cada tipo de geração, foram avaliados mapas gerados (Figuras 5 e 6) e fragmentos retirados desses mapas (Figuras 7, 8 e 9).

Na comparação dos fragmentos, foram observados os seguintes aspectos:

- a) Quantidade de Canos (Figura 7): Mapas gerados com o auxílio do algoritmo de seleção apresentaram posicionamento mais lógico de canos, comparando com as regras do jogo;

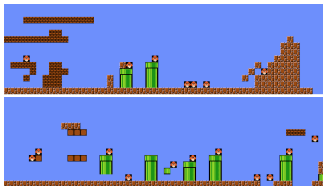


Figura 7: O primeiro fragmento é derivado da combinação da WGAN com o algoritmo de seleção, enquanto o segundo corresponde à WGAN pura, ambos com dificuldade média. Observa-se no caso da WGAN pura, a presença de inimigos mal posicionados e um excesso de elementos no cenário.

- b) Erros Estruturais (Figura 8): O algoritmo de seleção, por meio de aplicação de filtro e penalização, reduziu significativamente erros de posicionamento de canos e inimigos;



Figura 8: O primeiro fragmento foi gerado a partir da combinação da WGAN com o algoritmo de seleção, enquanto o segundo corresponde à WGAN pura, ambos com nível de dificuldade fácil. No caso da WGAN pura, observa-se a presença de inimigos mal posicionados, um excesso de canos e blocos, além de canos colocados de forma inadequada.

- c) Estruturas Ilógicas (Figura 9): Apesar das melhorias, ambos os métodos geraram algumas estruturas fora do padrão lógico do jogo. Isso indica que a rede generativa pode ter enfrentado dificuldades em aprender e reproduzir corretamente certas configurações estruturais.

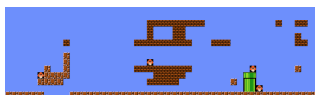


Figura 9: Fragmento gerado pelo algoritmo de seleção. Observa-se a presença de estruturas mal formadas e que não fazem sentido dentro da dinâmica do jogo.

C. Comparação entre os experimentos

A partir das análises, foi possível obter as seguintes conclusões:

- a) Tempo de Conclusão: Os resultados do experimento que utiliza algoritmo de seleção foram mais consistentes. Essa consistência se deve ao fato de que, em ambos grupos desse experimento, o desvio padrão foi baixo e a diferença entre eles foi consideravelmente menor ao se comparar com o primeiro experimento. Esses fatores

indicam que o algoritmo de seleção foi eficiente na seleção dos fragmentos.

- b) Qualidade dos Mapas: A análise empírica destacou que os mapas gerados exclusivamente com a WGAN apresentaram maior variedade, além de defeitos estruturais que não seguem os padrões do jogo. Enquanto aqueles refinados pelo algoritmo de seleção exibiram maior coesão, consistência e jogabilidade.
- c) Eficiência de geração dos mapas: Ao se comparar o número absoluto de mapas gerados, o experimento da WGAN vence a comparação, porém o algoritmo de seleção se destaca pelo aproveitamento da geração e divisão equilibrada de dificuldade, utilizando apenas os fragmentos jogáveis. Esses fatores demonstram que o algoritmo de seleção possui potencial de gerar mapas bem construídos e variados em detrimento de uma possível diminuição na quantidade de mapas gerados.

Esses resultados confirmam que, no geral, a junção da WGAN ao algoritmo de seleção resultou em uma geração de mapas jogáveis com maior eficiência e qualidade. A combinação dessas técnicas não apenas otimizou o tempo de conclusão, como também garantiu que os mapas produzidos apresentassem uma estrutura mais consistente e fragmentos bem alinhados, atendendo melhor aos critérios de jogabilidade e *design* esperados.

V. CONCLUSÃO

Esse estudo investigou a geração de mapas para o jogo Super Mario Bros. utilizando Redes Generativas Adversárias (GANs). A metodologia envolveu a divisão dos mapas originais do jogo em amostras menores, a codificação das informações e a realização de diversos testes. Uma WGAN, um algoritmo de seleção e um agente A* foram implementados para avaliar a eficácia da WGAN na geração de mapas jogáveis e o impacto do algoritmo de seleção no processo de geração.

Os resultados demonstraram que a combinação da WGAN com um algoritmo de seleção melhora a consistência estrutural e a jogabilidade dos mapas gerados em comparação ao uso isolado da WGAN. O algoritmo de seleção, ao filtrar fragmentos jogáveis e bem estruturados, contribuiu para mitigar a geração de mapas contendo erros ou inconsistências estruturais. Essa abordagem resultou em uma maior proporção de mapas que seguem a lógica e os princípios de design do jogo Super Mario Bros. Exemplo disso é que, ao comparar os mapas contendo apenas fragmentos jogáveis, houve um aumento de 25% na taxa de fragmentos jogáveis e, no quesito tempo médio de avaliação pelo agente, houve uma redução de 15%.

No entanto, o algoritmo de seleção, embora melhore a qualidade dos mapas, também levou a uma redução na quantidade total de mapas gerados. Esse equilíbrio destaca a importância de novas pesquisas para otimizar a relação entre qualidade e quantidade de mapas gerados.

Além disso, ambas as abordagens resultaram em uma predominância de mapas classificados como de baixa dificuldade. Isso sugere que trabalhos futuros devem explorar métodos para melhorar a capacidade da WGAN de gerar mapas com uma

gama mais diversificada de níveis de dificuldade. Possíveis caminhos incluem o refinamento do processo de treinamento da rede, a incorporação de métricas de avaliação mais sofisticadas no algoritmo de seleção ou a investigação de arquiteturas GAN alternativas, mais adequadas para capturar as complexidades do design de níveis.

Este estudo contribui para os campos de geração de imagens e Geração Procedural de Conteúdo (PCG) ao propor um método para a geração de mapas estruturados e jogáveis para jogos de plataforma 2D. Os resultados destacaram o potencial das GANs, especialmente quando combinadas com mecanismos inteligentes de seleção, para a criação de conteúdo de jogo envolvente e diversificado.

VI. TRABALHOS FUTUROS

A metodologia e os resultados desse estudo apontam para duas direções principais de pesquisa futura. A primeira envolve a integração de um método evolutivo no processo de treinamento da WGAN. As inconsistências identificadas na geração indicam a necessidade de explorar algoritmos evolutivos para aprimorar o treinamento da rede, com o objetivo de gerar fragmentos que se assemelhem mais aos padrões dos mapas originais do jogo. Esses algoritmos podem refinar o conteúdo gerado de forma iterativa, aumentando a capacidade da rede de capturar a estrutura e as regras do jogo-alvo. A segunda direção trata da automatização da definição dos pesos utilizados, que foram definidos empiricamente no presente estudo. Investigações futuras devem considerar ainda o uso de métodos heurísticos, como algoritmos genéticos, ou ferramentas de otimização, como o Optuna, para automatizar esse processo, o que pode tornar a seleção de pesos mais eficiente e melhorar o desempenho do sistema. Além disso, sugere-se a exploração de arquiteturas avançadas, como a StyleGAN, para capturar padrões mais complexos, além da inclusão de *feedback* de jogadores reais para validar a qualidade dos mapas gerados.

REFERÊNCIAS

- [1] D. da Manhã, “4 importantes transformações tecnológicas nos jogos eletrônicos.” <https://www.dm.com.br/cotidiano/4-importantes-transformacoes-tecnologicas-nos-jogos-eletronicos-127327>, Aug 2023. [Accessed 19-02-2024].
- [2] M. d. L. Reis and K. d. O. Andrade, “Áreas de pesquisa e técnicas de inteligência artificial em jogos digitais,” *Revista Tecnológica Fatec Americana*, 2023.
- [3] M. S. Siqueira, “Esports e o ambiente acadêmico: uma perspectiva sobre o cenário esports,” *Revista UFF*, 2023.
- [4] E. S. Andrade, “Games e a indústria cultural: O impacto sociocultural dos jogos eletrônicos,” *Repositório Institucional - Universidade Federal de Uberlândia: Home*, Jan 2023.
- [5] R. P. T. Machado, “Jogos eletrônicos e e-sports: Um fenômeno cultural,” <https://ciencia.ufba.br/todas-opinioes/860-jogos-eletronicos-e-e-sports-um-fenomeno-cultural>, Aug 2022. [Accessed 19-02-2024].
- [6] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” *Commun. ACM*, vol. 63, p. 139–144, Oct 2020.
- [7] J. Cheigh, “Generating images using vaes, gans, and diffusion models,” <https://towardsdatascience.com/generating-images-using-vaes-gans-and-diffusion-models-48963ddeb2b2>, 2023. [Accessed 09-03-2024].
- [8] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, and S. Risi, “Evolving mario levels in the latent space of a deep convolutional generative adversarial network,” in *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '18*, (New York, NY, USA), p. 221–228, Association for Computing Machinery, 2018.
- [9] M. Awiszus, F. Schubert, and B. Rosenhahn, “Toad-gan: Coherent style level generation from a single example,” *ArXiv*, 2020.
- [10] D. L. David J. Malan, “Csci e-23a - cs50,” <https://cs50.harvard.edu/extension/games/2021/fall/syllabus/>, 2021. [Accessed 09-03-2024].
- [11] G. Aloupis, E. D. Demaine, A. Guo, and G. Viglietta, “Classic nintendo games are (computationally) hard,” *Theoretical Computer Science*, vol. 586, pp. 135–160, 2015. Fun with Algorithms.
- [12] J. deWinter, *Shigeru Miyamoto: Super Mario Bros., Donkey Kong, The Legend of Zelda*. Influential Video Game Designers, Bloomsbury Publishing, 2015.
- [13] N. Shaker, J. Togelius, G. N. Yannakakis, B. Weber, T. Shimizu, T. Hashiyama, N. Sorenson, P. Pasquier, P. Mawhorter, G. Takahashi, G. Smith, and R. Baumgarten, “The 2010 mario ai championship: Level generation track,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 4, pp. 332–347, 2011.
- [14] A. Khalifa, “Mario-ai-framework,” <https://github.com/amidos2006/Mario-AI-Framework?tab=readme-ov-file#copyrights>, 2021. [Accessed 27-07-2024].
- [15] D. P. Kingma and M. Welling, “An introduction to variational autoencoders,” *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, p. 307–392, 2019.
- [16] D. F. e Silva, R. Torchelsen, and M. Aguiar, “Dungeon level generation using generative adversarial network: an experimental study for top-down view games,” in *Anais do L Seminário Integrado de Software e Hardware*, (Porto Alegre, RS, Brasil), pp. 95–106, SBC, 2023.
- [17] T. Shaham, T. Dekel, and T. Michaeli, “Singan: Learning a generative model from a single natural image,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, (Los Alamitos, CA, USA), pp. 4569–4579, IEEE Computer Society, Nov 2019.
- [18] J. Togelius, S. Karakovskiy, and R. Baumgarten, “The 2009 mario ai competition,” in *IEEE Congress on Evolutionary Computation*, pp. 1–8, 2010.
- [19] J. Schrum, J. Gutierrez, V. Volz, J. Liu, S. Lucas, and S. Risi, “Interactive evolution and exploration within latent level-design space of generative adversarial networks,” in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference, GECCO '20*, (New York, NY, USA), p. 148–156, Association for Computing Machinery, 2020.
- [20] A. Sarkar, Z. Yang, and S. Cooper, “Controllable level blending between games using variational autoencoders,” *ArXiv*, 2020.
- [21] A. Sarkar and S. Cooper, “Sequential segment-based level generation and blending using variational autoencoders,” *ArXiv*, 2020.
- [22] A. J. Summerville, S. Snodgrass, M. Mateas, and S. O. Villar, “The VGLC: The video game level corpus,” *CoRR*, vol. abs/1606.07487, 2016.
- [23] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 214–223, PMLR, 06–11 Aug 2017.
- [24] J. Gutierrez and J. Schrum, “Generative adversarial network rooms in generative graph grammar dungeons for the legend of zelda,” *ArXiv*, 2020.
- [25] J. Brownlee, “How to Develop a Wasserstein Generative Adversarial Network (WGAN) From Scratch - MachineLearningMastery.com — machinelearningmastery.com,” <https://machinelearningmastery.com/how-to-code-a-wasserstein-generative-adversarial-network-wgan-from-scratch/>, Jan 2021. [Accessed 27-07-2024].
- [26] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of wasserstein gans,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, (Red Hook, NY, USA), p. 5769–5779, Curran Associates Inc., 2017.

Centro Federal de Educação Tecnológica de Minas Gerais

Curso de Engenharia de Computação

Avaliação do Trabalho de Conclusão de Curso

Aluno: Victor Le Roy Matos

Título do trabalho: Geração de mapas para jogos em 2D utilizando Redes Generativas Adversariais.

Data da defesa: 10/02/2025

Horário: 14:00

Local da defesa: <https://conferenciaweb.rnp.br/webconf/rogerio-martins-gomes>

O presente Trabalho de Conclusão de Curso foi avaliado pela seguinte banca:

Professor Rogério Martins Gomes – Orientador

Departamento de Computação

Centro Federal de Educação Tecnológica de Minas Gerais

Professor João Gabriel Gama Vila Nova – Coorientador

Departamento de Computação

UNIMA Afya

Professor André Rodrigues da Cruz – Membro da banca de avaliação

Departamento de Computação

Centro Federal de Educação Tecnológica de Minas Gerais

Professor Bruno André Santos – Membro da banca de avaliação

Departamento de Computação

Centro Federal de Educação Tecnológica de Minas Gerais